

Codebook for:

The *Skyscraper Revolution*:

Global Economic Development and Land Savings

ABSJ2027 toolkit

Gabriel M. Ahlfeldt, Nathaniel Baum-Snow, and Rémi Jedwab

9 September 2026

Abstract

This codebook describes the MATLAB toolkit accompanying *The Skyscraper Revolution*. It defines the primitives and endogenous objects, states the equilibrium conditions, and gives the numerical algorithms in pseudocode. The toolkit quantifies a stylized city from population, urbanization, and an optional height-gap target. Users can then change fundamentals or parameters and compare the baseline and counterfactual equilibria. A horizontal growth boundary extends the paper's model and is inactive by default. The presentation is intended to support a subsequent Python implementation of the same algorithms; the current implementation is MATLAB.

Implementation companion. Monetary quantities are in model units unless an annual wage target supplies a currency conversion. The toolkit covers illustrative city experiments; it does not reproduce the complete empirical pipeline.

A The ABSJ2027 toolkit

A.1 Equilibrium

The economy consists of a circular city and its rural hinterland. Let $u \in \{R, C\}$ denote residential and commercial use, and let $x \geq 0$ denote distance in kilometres from the centre. Available land in a radial interval is $dL(x) = 2\pi\ell x dx$. The equilibrium state is wage y , urban utility $\bar{U}$, and density $\mathcal{D}$. Regional population $\bar{N}$ and the rural-utility primitive $\tilde{U}$ govern urban population:

$$N = \bar{N} \frac{\bar{U}^\zeta}{\bar{U}^\zeta + \tilde{U}}, \quad \mathcal{D} = \frac{N}{\pi\ell x_1^2}. \quad (1)$$

Rural utility itself is $\tilde{U}^{1/\zeta}$. Thus the urban-population elasticity with respect to urban utility is $\zeta(1 - N/\bar{N})$.

The height-independent productivity and amenity shifters are

$$A^C(x) = a_{\text{loc}}^C(x) \bar{a}^C N^{\beta_C} e^{-\tau^C \max\{0, x - \underline{x}^C\}}, \quad (2)$$

$$A^R(x) = a_{\text{loc}}^R(x) \bar{a}^R N^{\beta_P} \mathcal{D}^{\beta_D} e^{-\tau^R \max\{0, x - \underline{x}^R\}}. \quad (3)$$

Here β_D is the paper's β^R , called `beta_dens_R` in MATLAB. The additional population elasticity β_P is `beta_R`; its default is zero. Local multipliers equal one in the stylized baseline.

It is useful to define the demand shifters appearing directly in the code:

$$q^C(x) = A^C(x) \frac{1}{1 - \alpha^C} y^{-\frac{\alpha^C}{1 - \alpha^C}}, \quad (4)$$

$$q^R(x) = \left[\frac{A^R(x)y}{\bar{U}} \right]^{\frac{1}{1 - \alpha^R}}. \quad (5)$$

The variables `a_x_C` and `a_x_R` store q^C and q^R . These equal $(1 + \omega^u)$ times the paper's height-invariant average floor-space rent shifter. They exclude the height component of amenities or productivity.

A developer's construction cost is $c^u S^{1+\theta^u}$. Optimal unconstrained height, capped height, and the associated land bid satisfy

$$S^{u,*}(x) = \left[\frac{q^u(x)}{c^u(1 + \theta^u)} \right]^{\frac{1}{\theta^u - \omega^u}}, \quad \tilde{S}^u(x) = \min\{S^{u,*}(x), \bar{S}^u\}, \quad (6)$$

$$r^u(x) = \frac{q^u(x)}{1 + \omega^u} \tilde{S}^u(x)^{1+\omega^u} - c^u \tilde{S}^u(x)^{1+\theta^u}. \quad (7)$$

The restriction $\theta^u > \omega^u$ ensures a finite unconstrained height.

A.1.1 Land allocation and market clearing

Within the permitted urban region, land is assigned to the highest bid among $r^C(x)$, $r^R(x)$, and agricultural rent r^a . Write $\mathcal{C}$, $\mathcal{R}$, and $\mathcal{A}$ for the resulting land-use sets. The growth boundary $x_{1,\max}$ imposes agricultural use whenever $x > x_{1,\max}$. It is disabled when $x_{1,\max} = \infty$.

The commercial boundary x_0 separates the CBD and the residential zone; x_1 is the outermost residential radius. In the continuous model, an interior x_0 satisfies $r^C(x_0) = r^R(x_0)$. An unconstrained fringe satisfies $r^R(x_1) = r^a$. At a binding growth boundary, the residential land bid may exceed r^a . The growth limit must therefore enter land allocation before aggregates and equilibrium updates are calculated. It is not an ex-post truncation of a solved city.

Realized height S^u equals $\tilde{S}^u$ on land assigned to use u . Sectoral integrals below exclude other land. Average floor-space rent is

$$\bar{p}^u(x) = \frac{q^u(x)}{1 + \omega^u} S^u(x)^{\omega^u}. \quad (8)$$

Employment, housing per resident, and residents in each interval obey

$$L(x) dx = \frac{\alpha^C}{1 - \alpha^C} \frac{\bar{p}^C(x) S^C(x)}{y} dL(x), \quad (9)$$

$$\bar{f}^R(x) = \frac{(1 - \alpha^R)y}{\bar{p}^R(x)}, \quad n(x) dx = \frac{S^R(x)}{\bar{f}^R(x)} dL(x). \quad (10)$$

Market clearing requires $\int_{\mathcal{C}} L(x) dx = N = \int_{\mathcal{R}} n(x) dx$. Conditional on the current guesses, the solver constructs the following predictions:

$$\hat{y} = \frac{\alpha^C}{(1 - \alpha^C)N} \int_{\mathcal{C}} \bar{p}^C(x) S^C(x) dL(x), \quad (11)$$

$$J^R = \int_{\mathcal{R}} A^R(x)^{\frac{1}{1-\alpha^R}} S^R(x)^{1+\omega^R} dL(x), \quad (12)$$

$$\hat{U} = \left[\frac{y^{\frac{1}{1-\alpha^R}} J^R}{(1 + \omega^R)(1 - \alpha^R)yN} \right]^{1-\alpha^R}, \quad \hat{\mathcal{D}} = \frac{N}{\pi \ell x_1^2}. \quad (13)$$

The equilibrium routine iterates all three state variables. The standard concentric specification requires $\tau^C/(1 - \alpha^C) > \tau^R/(1 - \alpha^R)$. Arbitrary spatial multipliers can violate this structure; disconnected urban systems are outside the documented scope of this toolkit.

A.2 Codebook of inputs and objects

Table 1: Structural parameters and fundamentals

Object	MATLAB field	Meaning and default
α^R, α^C	alpha_R, alpha_C	Non-housing spending and labour shares: 0.66, 0.85.
β_D	beta_dens_R	Density elasticity of amenities: -0.1053554548 .
β_P, β^C	beta_R, beta_C	Population elasticities: 0, 0.03.
τ^R, τ^C	tau_R, tau_C	Spatial decay per km: 0.016, 0.014.
$\underline{x}^R, \underline{x}^C$	x_core_R, x_core_C	Flat core radii: 1, 1 km.
ω^R, ω^C	omega_R, omega_C	Height elasticities of rent: 0.07, 0.03.
θ^R, θ^C	theta_R, theta_C	Height elasticities of unit cost: 0.55, 0.50.
c^R, c^C	c_R, c_C	Construction-cost scales: 150, 150.
$\bar{a}^R, \bar{a}^C$	a_bar_R, a_bar_C	Fundamental amenity/productivity: 2, 2.
r^a	r_a	Agricultural land rent: 50.
ℓ	ell	Developable land share: 0.5.
ζ	zeta	Preference heterogeneity: 6.4; requires $\zeta > 0$.
T	T	Tall-height threshold: 2.9605069279.
$\bar{S}^R, \bar{S}^C$	S_bar_R, S_bar_C	Height limits: ∞, ∞ .
$x_{1,\max}$	x1_max	Maximum urban radius: ∞ km.
$a_{\text{loc}}^R, a_{\text{loc}}^C$	a_rand_R, a_rand_C	Positive local multipliers in <code>fund</code> ; default vectors of ones.
$\bar{N}$	N_bar	Regional population, calibrated to N^{target}/μ .
$\tilde{U}$	U_tilde	Rural utility to the power ζ , inverted to match μ .

Notes: Defaults preserve replication values, including $1 - \alpha^R = 0.34$ rather than the rounded housing share 0.33 in the paper. Estimated constants are stored at full precision in MATLAB. The default $\theta^R = \theta^C + 0.05$ initializes the baseline; subsequent changes to these fields are independent. Heights and T are effective floor-area ratios (floor space per unit developable land), not literal storey counts. Construction-cost c^u is the coefficient multiplying $S^{1+\theta^u}$ in the code. Exact closed-city $\zeta = 0$ is not supported.

Table 2: Empirical targets and equilibrium objects

Object	MATLAB name	Description
<i>Empirical targets</i>		
N^{target}	population	Urban population: 3,000,000.
μ	urban_share	Urban share of regional population: 0.5, strictly between zero and one.
g	height_gap	Unrealized tall floor-space fraction: 0; [] retains direct sector caps.
<i>Equilibrium state and local objects</i>		
$y, \bar{U}$	y, U_bar	Wage and urban utility.
$\mathcal{D}$	density	Urban population per developable square kilometre.
N	N	Endogenous urban population.
x_0, x_1	x0, x1	CBD boundary and residential fringe (km).
q^R, q^C	a_x_R, a_x_C	Demand shifters, Eqs. (4)–(5).
S^R, S^C	S_x_R, S_x_C	Realized sectoral building heights.
$\bar{p}^R, \bar{p}^C$	p_bar_x_R, p_bar_x_C	Local average floor-space rents.
r^R, r^C	r_x_R, r_x_C	Potential land bids by sector.
$\mathcal{C}, \mathcal{R}, \mathcal{A}$	U	Land-use codes: commercial 1, residential 2, agricultural 3.
<i>Selected aggregate objects</i>		
F^R, F^C	F_R, F_C	Sectoral floor space, $\int_u S^u dL$.
H	F_tallE	Floor space exceeding threshold T , Eq. (22).
GDP	GDP	Urban output yN/α^C .
V	V	Welfare including city and hinterland.
LV	LV	Total land rent over the numerical region.

Conditional on the construction-cost level, the three targets identify regional population, rural utility, and, when requested, a common height cap. An additional residential floor-space target can identify the cost level, as described in Section A.7. They do not separately estimate productivity or amenity fundamentals. Editing baseline inputs and rerunning quantifies a new baseline. A counterfactual instead preserves the calibrated primitives except for explicit changes, allowing urban population to respond.

Wages, rents, and output are expressed in model units. Neither these targets nor the toolkit supply an independent currency conversion.

A.3 Aggregate outcomes

GDP is urban output, rather than the wage bill. Welfare and total land rent include the rural hinterland:

$$\text{GDP} = \frac{yN}{\alpha^C}, \quad \text{wage bill} = yN, \quad (14)$$

$$V = (\tilde{U} + \bar{U}^\zeta)^{1/\zeta}, \quad (15)$$

$$\text{LV} = \int_{\mathcal{C}} r^C \, dL + \int_{\mathcal{R}} r^R \, dL + \int_{\mathcal{A}} r^a \, dL. \quad (16)$$

The last integral extends to the fixed numerical radius, 100 km by default. This radius is distinct from the urban growth boundary and should remain unchanged across baseline and counterfactual.

Define resident- and employment-weighted averages by

$$\langle z \rangle_R = \frac{\int_{\mathcal{R}} z(x) n(x) \, dx}{\int_{\mathcal{R}} n(x) \, dx}, \quad \langle z \rangle_C = \frac{\int_{\mathcal{C}} z(x) L(x) \, dx}{\int_{\mathcal{C}} L(x) \, dx}. \quad (17)$$

The reporting functions retain the following replication statistics:

$$p_R = \langle \bar{p}^R \rangle_R, \quad p_C = \langle \bar{p}^C \rangle_C, \quad (18)$$

$$\text{CC} = \langle e^{\tau^R x} \rangle_R, \quad \text{DD} = \mathcal{D}^{\beta_D}, \quad (19)$$

$$\text{HA} = \langle (S^R)^{\omega^R(1-\alpha^R)} \rangle_R. \quad (20)$$

The commuting-cost measure is an index, not a monetary cost or travel time. It uses x , as in the replication, rather than the core-adjusted distance in Eq. (3). Changes in these weighted mean indices do not provide an exact finite-change decomposition of utility; utility is solved from equilibrium.

Sectoral floor space is $F^u = \int_u S^u \, dL$. Two tall-floor-space measures must be distinguished:

$$F_{\text{tall}} = \sum_{u \in \{R, C\}} \int_u S^u(x) \mathbf{1}\{S^u(x) > T\} \, dL(x), \quad (21)$$

$$H = F_{\text{tallE}} = \sum_{u \in \{R, C\}} \int_u \max\{S^u(x) - T, 0\} \, dL(x). \quad (22)$$

The height-gap inversion uses the second measure. If H^* is its value after removing both height limits while retaining all other calibrated fundamentals, then

$$g = 1 - H/H^*. \quad (23)$$

If $H^* = 0$, the toolkit records zero with an identification flag and rejects a positive height-gap target.

A.4 Algorithms

The pseudocode below is language-independent. Function names map directly to separate files in MATLAB/functions. Guesses are inputs to an equilibrium evaluation; predictions are the values implied by that evaluation. Warm starts pass a previously solved state explicitly.

A.4.1 City creation and conditional equilibrium

Algorithm 1: Create the numerical city: CREATE_CITY

Data: Numerical radius R_{grid} , spacing h , baseline parameters.

- 1 Create symmetric coordinates $\mathbf{x} = -R_{\text{grid}} : h : R_{\text{grid}}$
- 2 Retain non-negative distances D for model computations
- 3 Set $\delta = (\max \mathbf{x} - \min \mathbf{x}) / \#\mathbf{x}$
- 4 Compute geographic cell areas $G_i = 2\pi\delta(D_i + \delta/2)$
- 5 Initialize $a_{\text{loc},i}^R = a_{\text{loc},i}^C = 1$

Result: fund: coordinates, geographic cell areas, local multipliers.

The grid defaults to $R_{\text{grid}} = 100$ km and $h = 0.01$ km. The replication uses δ , rather than h , in its quadrature weights. The toolkit preserves this convention. Developable cell area is ℓG_i ; reported urban area remains $\pi \ell x_1^2$.

Algorithm 2: Evaluate endogenous objects: SOLVER

Data: Primitives, **fund**, guesses $\{y, \bar{U}, \mathcal{D}\}$.

- 1 Compute N using Eq. (1)
- 2 Compute $A^u(x)$ and $q^u(x)$ using Eqs. (2)–(5)
- 3 Compute optimal and capped heights using Eq. (6)
- 4 Compute land bids using Eq. (7)
- 5 Assign each location to the use with the highest land bid
- 6 Impose agricultural use at every $D_i > x_{1,\text{max}}$
- 7 Recover x_0 and the outermost residential radius x_1
- 8 Restrict realized heights and floor-space rents to occupied sectors
- 9 Compute floor space, employment, and residents using Eqs. (9)–(10)
- 10 Compute $\hat{y}$, $\hat{\bar{U}}$, and $\hat{\mathcal{D}}$ using Eqs. (11)–(13)
- 11 Compute welfare, land rent, and the remaining aggregate statistics

Result: State predictions, local profiles, and aggregate outcomes.

A.4.2 Equilibrium iteration

Algorithm 3: Find equilibrium: EQFIND

Data: Primitives, **fund**, numerical controls, optional initial state.

```

1 Set  $s = (y, \bar{U}, \mathcal{D})$  to the initial state, or  $(1, 0.2, 1000)$ 
2 for  $k = 1, \dots, K_{\max}$  do
3   Evaluate Algorithm 2 at  $s$ , obtaining predictions  $\hat{s}$ 
4   Clip each component:  $\hat{s}_j \leftarrow \max\{0.9s_j, \min\{1.1s_j, \hat{s}_j\}\}$ 
5   Set  $e_j = |\hat{s}_j - s_j| / \max\{|s_j|, 10^{-8}\}$  for all components
6   if  $\max_j e_j < \varepsilon_{\text{eq}}$  then
7     Check finite positive outcomes, both urban sectors, and the grid edge
8     return the current state  $s$  and its evaluated outcomes
9   Compute replication update weights  $\lambda_j(k)$ 
10  Update each component:  $s_j \leftarrow (1 - \lambda_j)s_j + \lambda_j\hat{s}_j$ 
11 Report nonconvergence as an error

```

Result: Equilibrium state, local profiles, aggregates, iteration count.

The default tolerance is $\varepsilon_{\text{eq}} = 0.001$ and $K_{\max} = 1000$. Default “rich” damping combines large early updates with smaller late updates. For component j , the weight is

$$\lambda_j(k) = A + \frac{b_j - A}{[1 + \exp\{-B(k - M)\}]^{1/v}}, \quad (24)$$

where $A = 0.98$, $B = 1$, $M = 8$, $v = 1$, and

$$b_y = \min\{0.01, e_y\}, \quad b_{\bar{U}} = 0.05, \quad b_{\mathcal{D}} = 0.25. \quad (25)$$

These weights, clipping, and the convergence criterion follow the replication. A returned state satisfies the numerical tolerance, not exact analytical equality. Iteration exhaustion never counts as convergence.

A.4.3 Inverting the urbanization target

Conditional on $\bar{U}$, the rural primitive that rationalizes target share μ is

$$\tilde{U}_{\text{new}} = \bar{U} \zeta \frac{1 - \mu}{\mu}. \quad (26)$$

Because $\bar{U}$ responds to $\tilde{U}$, inversion requires an outer iteration. Algorithm 4 returns the primitive actually used in its equilibrium. This avoids the replication routine’s extra final update after reaching the target.

Algorithm 4: Invert rural utility: FINDU $\tilde{t}$ lde

Data: Primitives, fund, target μ , optional initial equilibrium.

- 1 **for** $k = 1, \dots, K_{\max}$ **do**
- 2 Solve equilibrium at the current $\tilde{U}$ using Algorithm 3
- 3 Compute the implied share $\hat{\mu} = N/\bar{N}$
- 4 **if** $|\hat{\mu} - \mu|/\mu < \varepsilon_{\mu}$ **then**
- 5 **return** the current $\tilde{U}$ and its corresponding equilibrium
- 6 Compute $\tilde{U}_{\text{new}}$ using Eq. (26)
- 7 Dampen the rural-utility update using the replication weights
- 8 Retain the solved equilibrium as the next warm start
- 9 Report failure to match the urban share

Result: Calibrated rural primitive and equilibrium at that primitive.

The default share tolerance is $\varepsilon_{\mu} = 0.001$. Rural inversion uses Eq. (24) with $A = 0.99$, $M = 22$, $B = v = 1$, and the state-dependent late weight $b = 0.5/[1 + \exp\{1 - 1000|\tilde{U}_{\text{new}} - \tilde{U}|\}]$. This outer iteration leaves other primitives fixed.

A.4.4 Inverting the height gap

Algorithm 5: Height-gap objective: HG_obj

Data: Candidate common cap H_L , target gap g , target share μ , primitives.

- 1 Reject invalid candidates with $H_L < T$ or $H_L \leq 0$
- 2 Set $\bar{S}^R = \bar{S}^C = H_L$
- 3 Use Algorithm 4 to match μ and record $H = F_{\text{tallE}}$
- 4 Set both height caps to ∞ in a copy of the calibrated primitives
- 5 Retain $\tilde{U}$, $\bar{N}$, construction parameters, and $x_{1,\max}$
- 6 Solve the unrestricted-height equilibrium and record H^*
- 7 **if** $H^* > 0$ **then**
- 8 Compute $\hat{g} = 1 - H/H^*$
- 9 **else**
- 10 Set $\hat{g} = 0$ and flag the gap as unidentified

Result: Absolute objective $|\hat{g} - g|$, simulated gap, identification status.

Here “unrestricted” refers only to height. The same horizontal growth boundary applies to the constrained and unrestricted-height cities. Construction-cost elasticities remain independently specified throughout the inversion.

Algorithm 6: Quantify at a given cost level: `QUANTIFY_CITY`

Data: Primitives, `fund`, N^{target} , μ , optional gap target g .

- 1 Set $\bar{N} = N^{\text{target}}/\mu$
- 2 Initialize $\tilde{U} = 0.2^\zeta(1 - \mu)/\mu$
- 3 **if** *either supplied sector cap is finite, or the gap target is empty* **then**
- 4 | Retain both sector-specific caps and skip height-gap matching
- 5 **else if** $g = 0$ **then**
- 6 | Set $\bar{S}^R = \bar{S}^C = \infty$
- 7 **else if** $g = 1$ **then**
- 8 | Set $\bar{S}^R = \bar{S}^C = T$
- 9 **else**
- 10 | Minimize Algorithm 5 over H_L using `fminsearch`
- 11 | Set both sector caps to the selected H_L
- 12 At the chosen caps, invert rural utility with Algorithm 4
- 13 Verify population and, when not overridden, the height-gap target
- 14 Package the calibrated inputs, solved equilibrium, and diagnostics

Result: Baseline city with calibrated fundamentals and saved outcomes.

The absolute height-gap tolerance is 0.01 in fractional units. Interior-gap optimization uses a cap tolerance of 0.01, at most 250 iterations, and at most 300 objective evaluations. Exact zero and one targets use the endpoints above, replacing the replication’s approximate 1% and 99% shortcuts. Positive targets are rejected when unrestricted tall floor space is zero.

A.4.5 Counterfactual solution

A counterfactual starts from the quantified baseline. Absolute changes replace values; percentage changes apply to the saved baseline value. Regional population and rural utility remain fixed unless explicitly changed. Baseline empirical targets are not re-inverted.

Algorithm 7: Change primitives and solve: `APPLY_CHANGES`, `COUNTERFACTUAL`

Data: Saved baseline and optional absolute or percentage changes.

- 1 **if** *no changes are supplied* **then**
- 2 | **return** an empty counterfactual
- 3 Copy the calibrated baseline primitives and local fundamentals
- 4 Reject unknown fields and fields included in both change operations
- 5 Apply each absolute change by replacement
- 6 Apply each percentage change as $v_{\text{new}} = v_{\text{baseline}}(1 + X/100)$
- 7 Reject percentage changes to zero or infinite baseline values
- 8 Validate the changed inputs jointly
- 9 Solve with Algorithm 3, using the baseline as a warm start

Result: Counterfactual equilibrium, without recalibrating baseline targets.

A.4.6 Reporting and exports

Algorithm 8: Package and display results: `PACK_RESULT`, `RESULTS`, `SAVE_RESULTS`

Data: Baseline and optional counterfactual equilibrium.

- 1 Add GDP, wage bill, density, density amenity, urban share, and total floor space to saved aggregates
- 2 Record target fit and market-clearing diagnostics
- 3 Build a table with baseline levels and units
- 4 **if** *a counterfactual is present* **then**
 - 5 Append counterfactual levels
 - 6 Compute $100(v_{CF}/v_{\text{baseline}} - 1)$ where defined
- 7 Plot saved heights, floor-space rents, and land bids
- 8 Show land-use zones and boundaries for each scenario
- 9 **if** *export is requested* **then**
 - 10 Save the table as CSV, results as MAT, and figures as PNG

Result: Summary table, figure handles, and optional exported files.

The graph wrappers `GHEIGHT`, `GBIDRENT`, and `GLANDRENT` call `PLOT_PROFILE`. They plot saved equilibrium profiles, preserving the user's local fundamentals. They do not invoke a new solver evaluation. Height and floor-space rent are shown on occupied sectoral land; land bids show potential bids by sector. An empty counterfactual produces baseline-only figures and statistics.

A.5 Using the master script

Run `MASTER_PAPER.m` or `MASTER_EMPIRICAL.m` in `MATLAB/scripts`; `MASTER.m` delegates to the paper version. The user-facing inputs are baseline parameters and fundamentals, empirical moments, and optional counterfactual changes. The execution sequence is

$$\text{CREATE_CITY} \longrightarrow \text{QUANTIFY} \longrightarrow [\text{COUNTERFACTUAL}] \longrightarrow \text{RESULTS}.$$

For example, to increase productivity and reduce residential construction cost:

```
changes = struct();           % Leave empty for baseline only.
changes.pct.a_bar_C = 10;    % Productivity increases by 10%.
changes.pct.c_R = -10;       % Residential cost falls by 10%.
```

Each change is relative to the saved baseline, so repeated solution does not compound it. Reducing c^R lowers construction cost at every height; changing θ^R changes how unit cost increases with height. These are distinct experiments.

To combine an urban growth boundary and height constraints:

```
changes = struct();
changes.set.x1_max = 25;    % Maximum urban radius in km.
changes.set.S_bar_C = 5;    % Commercial effective-height cap.
changes.set.S_bar_R = 5;    % Residential effective-height cap.
```

Use absolute values for limits initially equal to infinity, zero-valued parameters, and elasticities for which a percentage change would be ambiguous. No field may appear in both `set` and `pct`. Any finite `params.S_bar_C` or `params.S_bar_R` overrides `targets.height_gap`; both sector limits are then retained as entered. With both caps infinite, a numeric gap is matched and an empty gap leaves the city unrestricted. These height limits are effective floor-area ratios: a limit of five permits five square metres of floors per square metre of developable land, not necessarily five storeys.

A.6 Implementation and validation

A.6.1 Relationship to the replication code

The toolkit preserves the core solver's arithmetic and aggregation order. The growth-boundary extension assigns $U(D > \text{params.x1_max}) = 3$ before boundaries and aggregates are calculated. It does not set the developable share ℓ to zero. The two originally hard-coded one-kilometre core radii are parameters with defaults of one. With these defaults and `x1_max = Inf`, the original conditional solver is recovered.

The equilibrium routine preserves clipping and damping while replacing implicit workspace state with explicit arguments. Rural inversion checks the target before its final update, so the returned primitive belongs to the returned equilibrium. Its starting value is consistent with the target share. Baseline quantification does not depend on a preceding estimation run.

`MASTER.m` contains no function definitions. All named functions reside in individual files in `MATLAB/functions`; anonymous optimization handles only bind arguments to separate named objective functions. The toolkit omits Stata preprocessing, empirical estimation and bootstrap batches, all-city loops, and global aggregation. User population and urbanization targets are not silently capped. Changing structural parameters does not re-estimate other constants.

A.6.2 Numerical checks and limits

`VALIDATE_INPUTS` checks parameter domains, the ordering of spatial gradients, and change syntax. `EQFIND` rejects nonconvergence, invalid equilibria, missing commercial or residential sectors, and an unconstrained city at the grid edge. A larger grid requires baseline recalibration; it should not change between scenarios because it determines the agricultural land-rent domain.

Table 3: Function inventory

Files in MATLAB/functions	Role
CREATE_CITY	Construct coordinates, integration weights, and local fundamentals.
NUMERICS	Supply grid, iteration, damping, and inversion controls.
VALIDATE_INPUTS	Check inputs and domains.
SOLVER	Evaluate the model conditional on the state.
EQFIND	Iterate to equilibrium.
FINDUtilde	Invert the urbanization moment.
HG_obj	Evaluate the height-gap objective.
QUANTIFY	Dispatch baseline calibration with or without a floor-space target.
QUANTIFY_CITY, MATCH_FLOORSPACE	Match city moments conditional on cost, and calibrate a common cost multiplier to floor space.
APPLY_CHANGES	Apply absolute or percentage changes.
COUNTERFACTUAL	Solve the changed city.
PACK_RESULT	Add derived outcomes and diagnostics.
RESULTS	Create summary statistics and figures.
GHEIGHT, GBIDRENT, GLANDRENT	Select height, floor-space rent, or land-bid profiles.
PLOT_PROFILE	Draw profiles and land-use boundaries.
SAVE_RESULTS	Export CSV, XLSX, TEX, MAT, PNG, and PDF files.
WRITE_TABLE_TEX, TEX_ESCAPE	Format the exported LaTeX table and its labels.
REPORT_UNITS, REPORT_CITY	Convert displayed units and compute derived reporting statistics.
STATUS	Display numerical progress, with optional quiet mode.
VALIDATE_TOOLKIT	Run the toolkit’s regression and behavior checks.

The validation suite covers the baseline-only master, graphics, GDP, moment and market fit, unchanged counterfactuals, all three illustrative paper experiments, joint horizontal and vertical limits, interior height-gap inversion, direct-cap mode, independent parameter changes, and invalid-input rejection. With an optional read-only replication path, it compares isolated reference copies against the original solver and baseline inversion.

Eight validation groups passed in MATLAB R2024a Update 2. At the tested unrestricted state, original and toolkit conditional solver outputs were exactly identical; baseline inversion also matched exactly. The maximum difference from the rounded illustrative reference table was 0.095 percentage points, below the 0.6 acceptance threshold. These checks do not establish convergence for every possible parameter combination.

See `VALIDATION.md` for the recorded checks, `SOURCE_MAP.md` for provenance and implementation decisions, and `SOURCE_SHA256.json` for source hashes. MATLAB and the Statistics and Machine Learning Toolbox are required by the current implementation, which retains `nansum`. The Python directories are reserved for a later implementation of these algorithms.

A.7 Matching residential floor space

The optional `targets.floor_space_per_resident` targets residential square metres per model resident; its default is empty (disabled). Starting costs $c_0^R = c_0^C = 150$ imply about 956 square metres in the unrestricted baseline. A common multiplier k sets $c^R = kc_0^R$ and $c^C = kc_0^C$, preserving their initial ratio. An empty target retains both supplied costs. This calibrates physical quantities rather than changing reporting units.

At each conditional city solution, compute and predict an update:

$$\bar{f}_{\text{sim}} = 10^6 \frac{\int_{\mathcal{R}} \bar{f}^R(x) n(x) dx}{\int_{\mathcal{R}} n(x) dx}, \quad \frac{k_{\text{new}}}{k} = \left(\frac{\bar{f}_{\text{sim}}}{\bar{f}_{\text{target}}} \right)^{1+\theta^R}. \quad (27)$$

This residential scaling rule is a numerical prediction, not an exact joint-sector solution. Each candidate re-solves both sectors and re-matches the other active moments before checking the actual floor-space fit.

Algorithm 9: Match residential floor space: MATCH_FLOORSPACE

Data: Primitives, city grid, moments, positive $\bar{f}_{\text{target}}$.

- 1 Store starting costs c_0^R, c_0^C and initialize $k = 1$
- 2 **for** *at most 20 iterations* **do**
- 3 Set $c^R = kc_0^R, c^C = kc_0^C$ and run Algorithm 6
- 4 Compute resident-weighted $\bar{f}_{\text{sim}}$ in square metres
- 5 **if** $|\bar{f}_{\text{sim}}/\bar{f}_{\text{target}} - 1| < 0.001$ **then**
- 6 **return** the calibrated baseline and moment-fit diagnostics
- 7 Update k using the rule, bounding its change factor to $[0.01, 100]$
- 8 Retain density as a warm start; predict wage and utility responses to both sectors' cost changes
- 9 Report nonconvergence rather than accepting an unmatched target

Result: Baseline matching floor space and the other active moments.

Manual FAR limits supersede height-gap calibration. Counterfactuals inherit both calibrated costs; explicit changes remain independent. In the earlier 50-square-metre calibration example, $k \simeq 55.85$ gives about 50.04 square metres per resident. Height-gap calibration is disabled for this empirical quantification because the retained threshold T belongs to the paper parameterization. Thresholds are not silently changed to force a fit. Core equations are unchanged, but paper-reference checks disable this additional calibration.

A.8 Inverting the optional empirical primitives

The empirical example jointly fits residential rent (or floor-space consumption), total geographic urban area and central CBD FAR. Wage fixes monetary reporting units; it is not a fourth primitive in the search. An empty target omits its search coordinate and retains the supplied primitive. Height-gap inversion is disabled because T belongs to the paper parameterization; manual height limits remain applicable.

For active coordinates, the code uses $k_j = \exp(z_j - 1)$:

$$(c^R, c^C) = k_c(c_0^R, c_0^C), \quad r_a = k_a r_{a0}, \quad (\tau^R, \tau^C) = k_\tau(\tau_0^R, \tau_0^C). \quad (28)$$

Thus each common multiplier preserves its starting sectoral ratio. Coordinates are ordered housing, area, then CBD FAR. These primitives affect the moments jointly; the initialization rules below are numerical predictions, not separate analytical inversions.

Algorithm 10: Initialize optional primitives: MATCH_CITY_TARGETS

Data: Starting primitives $\mathcal{P}_0$, grid, population and urbanization targets; active optional moments m^* .

- 1 Reject simultaneous rent and consumption targets; require positive wage for rent
- 2 Require positive starting taus for CBD FAR; reject a CBD target at or above a finite manual commercial cap
- 3 Reject an area target outside the grid or growth boundary
- 4 Solve the starting city with Algorithm 6; save state $(y_s, \bar{U}_s, \mathcal{D}_s)$ and moments m_s
- 5 Create one coordinate per active multiplier and initialize each to one
- 6 **if** *housing is targeted* **then**
 - 7 Set $R = \bar{f}_s / \bar{f}^*$ for consumption, or $R = \bar{p}^{R*} / \bar{p}_s^R$ for rent
 - 8 Set $z_c^{(0)} = 1 + (1 + \theta^R) \log R$
- 9 **if** *area is targeted* **then**
 - 10 Set $z_a^{(0)} = 1 + \log(A_s / A^*)$
- 11 **if** *central CBD FAR is targeted* **then**
 - 12 Set preliminary city B to the starting city
 - 13 **if** *housing or area is also targeted* **then**
 - 14 Run Algorithm 12 with the CBD target temporarily inactive
 - 15 Set B to that fitted city; replace active coordinates by $1 + \log(c_B^R / c_0^R)$ and $1 + \log(r_{aB} / r_{a0})$
 - 16 Set $z_\tau^{(0)} = 1 + 0.8 \log(S^{C*} / S_B^C(0))$

Result: Starting vector $z^{(0)}$ and original solved state $(y_s, \bar{U}_s, \mathcal{D}_s)$.

The preliminary call has its CBD target switched off and cannot recurse again. Its fitted cost and land-rent levels seed the final search. The warm-start reference remains the original solved state. The exponent 0.8 is a starting-guess heuristic, not a calibrated model elasticity.

Evaluating a candidate city

Each objective evaluation applies multipliers to the original supplied primitives, not to the previous candidate. For numerical initialization only, define

$$h_R = k_c^{-1/(1+\theta^R)}, \quad h_y = k_c^{-(1-\alpha^C)(1+\omega^C)/(1+\theta^C)}. \quad (29)$$

The trial guesses are $y^{(0)} = y_s h_y$, $\bar{U}^{(0)} = \bar{U}_s h_y h_R^{(1+\omega^R)(1-\alpha^R)}$ and $\mathcal{D}^{(0)} = \mathcal{D}_s$. If housing is not targeted, $k_c = 1$. These guesses do not impose an equilibrium: the existing solver determines wages, utility and density.

Algorithm 11: Evaluate optional-target errors: CITY_TARGET_OBJECTIVE

Data: Candidate z , original primitives $\mathcal{P}_0$, original solved state, active targets.

- 1 Set inactive multipliers to one; set active $k_j = \exp(z_j - 1)$
- 2 Apply k_c to both construction costs, k_a to agricultural rent, and k_τ to both decay parameters
- 3 Construct wage, utility and density guesses using the formulas above
- 4 Run Algorithm 6: infer regional population from population/urbanization, invert rural utility and solve the equilibrium, retaining manual limits
- 5 **if** *a recognized trial equilibrium or inversion failure occurs* **then**
- 6 During search, return penalty $10^6 + \sum_j z_j^2$
- 7 During final verification, propagate the failure
- 8 Propagate unexpected programming or input errors
- 9 **if** *consumption is targeted* **then**
- 10 Compute $\bar{f} = 10^6 \sum_x n_x \bar{f}_x^R / \sum_x n_x$
- 11 **if** *rent is targeted* **then**
- 12 Compute $\bar{p}_{\text{currency}}^R = \bar{p}_{\text{model}}^R(w^*/y)/10^6$
- 13 **if** *area is targeted* **then**
- 14 Compute total geographic area $A = \pi x_1^2$
- 15 **if** *CBD FAR is targeted* **then**
- 16 Read central commercial FAR $S^C(0)$
- 17 Collect active simulated moments m_j in housing–area–CBD order
- 18 Compute $e_j = m_j/m_j^* - 1$ and $Q(z) = \sum_j [\log(m_j/m_j^*)]^2$
- 19 Report maximum relative error, both costs, agricultural rent and the tau multiplier

Result: Objective Q , candidate city and proportional moment errors e .

The sums use residential population weights and omit unoccupied cells. The model rent is the resident-weighted average from the solver. w^* is the observed annual wage, in the same currency as the rent target. Total area includes non-developable land; CBD FAR is a central value rather than a district-wide mean.

Updating the primitives and accepting the inversion

The current implementation uses MATLAB’s derivative-free Nelder–Mead search, `fminsearch`, rather than a Newton or Broyden update. It searches jointly over the active log coordinates. Reflection, expansion, contraction and shrink steps generate new candidates; each candidate invokes Algorithm 11. The iteration and evaluation limits bound computational work, not the positive primitive values themselves.

Algorithm 12: Joint primitive inversion: MATCH_CITY_TARGETS

Data: Original primitives, grid, numerical controls and active empirical targets.

- 1 Obtain $z^{(0)}$ and a solved reference state from Algorithm 10
- 2 Initialize the `fminsearch` simplex around $z^{(0)}$
- 3 **while** *the search has not stopped* **do**
 - 4 Evaluate proposed candidates with Algorithm 11
 - 5 Rank simplex vertices by Q
 - 6 Use Nelder–Mead reflection, expansion, contraction or shrink to update the simplex
 - 7 Stop early if the best $Q < [\log(1.002)]^2$
 - 8 Otherwise apply `TolX`= 0.0005 and `TolFun`= 10^{-9}
 - 9 Stop on reaching 200 iterations or 400 objective evaluations
- 10 Re-evaluate the returned z with Algorithm 11, allowing no failure penalty
- 11 **if** *any* $|e_j| > 0.005$ **then**
 - 12 Report failure rather than returning an unmatched baseline
- 13 Store the city, starting primitives, fitted multipliers and moment-fit diagnostics
- 14 Restore user numerical options rather than retaining internal warm-start guesses

Result: Verified empirical baseline; fitted primitives are inherited by counterfactuals unless explicitly changed.

`STOP_CITY_TARGET_SEARCH` implements the early-stop condition, which bounds each proportional moment error by approximately 0.2%. Final acceptance allows 0.5% because radial land-use boundaries are discrete. The preliminary and final searches each have their own iteration/evaluation limits; the starting solve and final verification are additional calls. A successful optimizer exit is not a substitute for the final moment checks.

Every candidate re-inverts rural utility and can require multiple equilibrium solves. Candidates currently reuse an adjusted version of the same original solved state, not the nearest previously solved candidate. Together with the preliminary fit and the empirical script’s finer grid, this explains why the optional inversion is slower than the paper baseline. Floor-space-only calibration without area or CBD targets continues to use Algorithm 9 and its 0.1% tolerance.

A.9 Central CBD FAR and the two master scripts

The optional `targets.cbd_far` identifies a common scale $k_\tau > 0$ such that

$$\tau^R = k_\tau \tau_0^R, \quad \tau^C = k_\tau \tau_0^C, \quad S^C(0) = S_{\text{target}}^C. \quad (30)$$

Both starting decay rates must be positive. The moment is realized commercial floor-area ratio at the city centre, measured as floor space per unit developable land, not the CBD-wide mean or a literal storey count. An empty target retains the supplied `taus`. A target at or above a finite manual commercial cap is rejected: it is either impossible or the cap prevents identification of the decay scale.

Algorithms 10–12 explicitly add one log coordinate and one log moment error when this target is active. For initialization, it first fits any housing and area targets at the supplied `taus`. It predicts a tau multiplier from the resulting central FAR mismatch, then jointly adjusts all active parameters. Every trial re-matches population and urbanization, retaining manual height limits. The final 0.5% check also applies to CBD FAR; no empirical moment is silently discarded. Diagnostics retain the starting `taus`, common multiplier and achieved central FAR. Counterfactuals inherit calibrated `taus` unless explicitly changed.

`MASTER_PAPER.m` preserves the paper’s parameter values and the original 10-metre radial grid. Population, urbanization and height-gap inversion remain active, while wage conversion and additional physical targets are empty. It saves figures and tables under `MATLAB/outputs/Paper`.

`MASTER_EMPIRICAL.m` also starts with the paper’s parameters, but targets annual wage 75,000 USD, residential rent 240 USD per square metre per year, geographic urban area $\pi 18^2$ square kilometres, and central CBD FAR 10. A 5-metre grid resolves boundaries under steeper spatial gradients. The script estimates the construction-cost scale, agricultural land rent and tau scale; it does not preset the fitted tau multiplier. It saves under `MATLAB/outputs/Empirical`.

Both scripts reset their input structures, accept optional counterfactual changes and contain no function definitions. `MASTER.m` is a compatibility entry point for the paper script. The separate output directories retain both examples when users switch between them. The empirical example is an alternative quantification, not a claim that the paper’s original parameterization matches those additional physical moments.

A.10 Height-gap inversion and the reported outcome

The estimated tall-height threshold T is tied to the paper’s parameterization. Height-gap inversion is therefore permitted only with the paper structural parameter values and no optional physical calibration. `IS_PAPER_PARAMETERS` checks these values. Regional population, rural utility and policy limits are not structural parameters in this check. A finite manual cap still supersedes an ignored height-gap input. The empirical master uses `targets.height_gap = []`; changing construction costs, spatial decay or agricultural rent does not recalibrate T .

Both master scripts nevertheless report the realized height gap as an outcome. For a scenario with primitives $\mathcal{P}$, let $H(\mathcal{P})$ denote its excess floor space above T , and let $\mathcal{P}^\infty$ remove only its vertical limits. Then

$$g(\mathcal{P}) = 1 - \frac{H(\mathcal{P})}{H(\mathcal{P}^\infty)}, \quad H(\mathcal{P}^\infty) > 0. \quad (31)$$

The table displays $100g$ in percent. Each scenario uses its own unrestricted-height comparison, with its other primitives, rural utility, regional population and growth boundary held fixed. If unrestricted tall development is absent, the gap is undefined and displayed as $-$; it is not recorded as zero. The inversion’s zero-target shortcut does not change this reporting convention.

`ADD_HEIGHT_GAP` reuses an available inversion benchmark, or solves the unrestricted comparison after the city has been quantified. An already unrestricted city is its own benchmark. `QUANTIFY` and `COUNTERFACTUAL` store the outcome; `RESULTS` upgrades older saved cities when needed. Optional calibration trials do not solve unused height-gap comparisons solely for reporting.

Under the empirical parameterization, the reported number is descriptive at the retained model T , and should not be interpreted as matching the paper’s empirical height-gap measure. The codebook’s earlier optional-calibration algorithms are subject to this restriction: they match the other requested moments and retain manual height limits, but do not invert a height-gap target.